

فایل جدید: 19-Java-Logging.md 

فاز ۱۹: لاگ‌گیری (Logging) در جاوا

لاگ‌گیری یکی از مهم‌ترین کارها برای Debugging، Monitoring و عیب‌یابی در برنامه‌های واقعی. توی مصاحبه ازت می‌پرسن با چه ابزاری کار کردی و چرا.

۱۹.۱ چرا لاگ؟

1. Debugging: پیدا کردن باگ‌ها

2. Monitoring: بررسی وضعیت برنامه

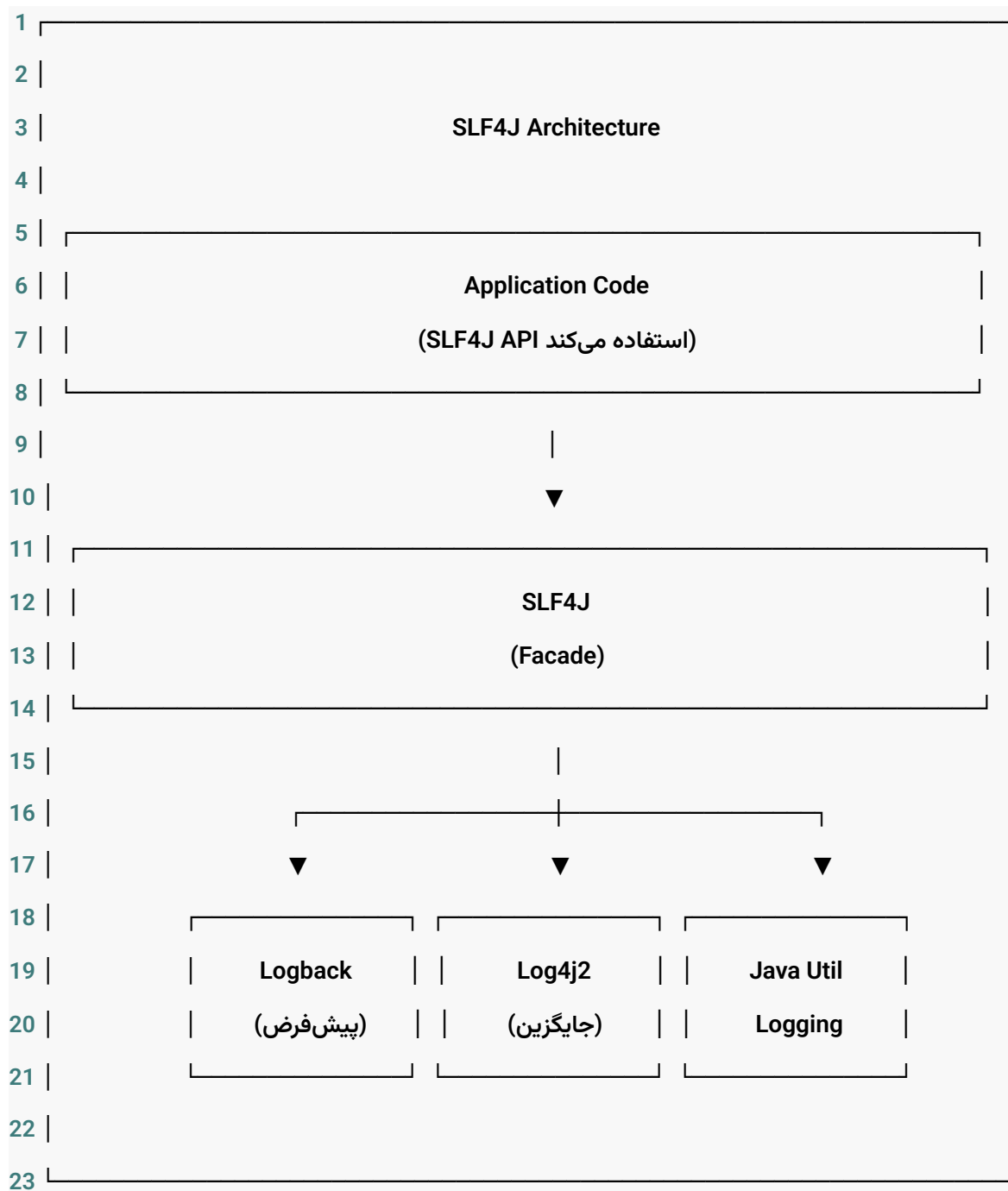
3. Audit: ثبت فعالیت‌های کاربران

4. Troubleshooting: عیب‌یابی مشکلات

5. Performance: بررسی عملکرد

۱۹.۲ SLF4J (Simple Logging Facade for Java)

SLF4J به Facade (رابط) برای لاگ‌گیری هست. یعنی خودش لاگ نمی‌نویسه، بلکه به کتابخانه‌های دیگه وصل میشه.



مزایای SLF4J

• وابستگی کم: می‌تونی هر کتابخانه‌ای رو استفاده کنی

• ساختار یکسان: فرقی نمی‌کند از Logback یا Log4j استفاده کنی، کد یکسانه

• پارامتریشدن: با {} راحت‌تر

۱۹.۳ Logback (پیش‌فرض Spring Boot)

تنظیمات در application.properties

```

1 # a. سطح لاگ برای پکیج‌ها
2 logging.level.root=INFO
3 logging.level.com.example=DEBUG
4 logging.level.org.springframework=INFO
5 logging.level.org.hibernate=WARN
6 logging.level.org.hibernate.SQL=DEBUG
7 logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
8
9 # b. فایل لاگ
10 logging.file.name=logs/myapp.log
11 logging.file.path=logs/
12
13 # c. فرمت لاگ
14 logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
15 logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
16
17 # d. گروه‌های لاگ (برای گروه‌بندی)
18 logging.group.db=org.hibernate,org.springframework.jdbc
19 logging.level.db=DEBUG

```

فایل logback-spring.xml (تنظیمات پیشرفته)

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3
4   <!-- ۱. Console Appender -->
5   <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
6     <encoder>
7       <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
8     </encoder>
9   </appender>
10
11  <!-- ۲. File Appender -->
12  <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
13    <file>logs/myapp.log</file>
14
15    <!-- چرخش فایل بر اساس زمان -->
16    <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
17      <fileNamePattern>logs/myapp.%d{yyyy-MM-dd}.log</fileNamePattern>
18      <maxHistory>30</maxHistory> <!-- روز نگهداری ۳۰ -->
19    </rollingPolicy>
20
21    <encoder>
22      <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
23    </encoder>
24  </appender>
25

```

```

26 <!-- ۳. File Appender با اندازه محدود -->
27 <appender name="FILE_SIZE" class="ch.qos.logback.core.rolling.RollingFileAppender">
28     <file>logs/myapp.log</file>
29     <rollingPolicy class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">
30         <fileNamePattern>logs/myapp.%d{yyyy-MM-dd}.%i.log</fileNamePattern>
31         <maxFileSize>10MB</maxFileSize>
32         <maxHistory>30</maxHistory>
33         <totalSizeCap>1GB</totalSizeCap>
34     </rollingPolicy>
35     <encoder>
36         <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
37     </encoder>
38 </appender>
39
40 <!-- ۴. JSON Formatter ( برای ELK ) -->
41 <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
42     <encoder class="net.logstash.logback.encoder.LogstashEncoder">
43         <includeMdc>true</includeMdc>
44     </encoder>
45 </appender>
46
47 <!-- ۵. تنظیم سطح لاگ برای پکیج های مختلف -->
48 <logger name="com.example" level="DEBUG"/>
49 <logger name="org.springframework" level="INFO"/>
50 <logger name="org.hibernate" level="WARN"/>
51 <logger name="com.example.security" level="DEBUG"/>

```

```

52
53 <!-- ۶. Root Logger -->
54 <root level="INFO">
55     <appender-ref ref="CONSOLE"/>
56     <appender-ref ref="FILE"/>
57 </root>
58
59 <!-- ۷. در محیط خاص JSON فعال کردن -->
60 <springProfile name="prod">
61     <root level="INFO">
62         <appender-ref ref="JSON"/>
63     </root>
64 </springProfile>
65
66 </configuration>

```

۱۹.۴ استفاده از SLF4J در کد

روش ۱: با Lombok (توصیه شده)

```

1 import lombok.extern.slf4j.Slf4j;
2
3 @Service
4 @Slf4j
5 public class UserService {
6

```

```
7 public void processUser(Long userId) {
8     // ۱. اطلاعات
9     log.info("Processing user: {}", userId);
10
11    // ۲. خطا
12    try {
13        // کد...
14    } catch (Exception e) {
15        log.error("Error processing user: {}", userId, e);
16    }
17
18    // هشدار ۳.
19    if (userId < 0) {
20        log.warn("Invalid user ID: {}", userId);
21    }
22
23    // دیباگ ۴.
24    log.debug("User processing started: {}", userId);
25
26    // جزئی‌ترین ۵. trace
27    log.trace("User processing step 1");
28
29    // بدون پارامتر ۶.
30    log.info("Method called");
31 }
32 }
```

روش ۲: بدون Lombok

```

1 import org.slf4j.Logger;
2 import org.slf4j.LoggerFactory;
3
4 @Service
5 public class UserService {
6     private static final Logger log = LoggerFactory.getLogger(UserService.class);
7
8     public void processUser(Long userId) {
9         log.info("Processing user: {}", userId);
10    }
11}

```

روش ۳: Dynamic Logging (با پارامترهای شرطی)

```

1 @Service
2 @Slf4j
3 public class UserService {
4
5     public void processUser(Long userId) {
6         // چک کردن سطح لاگ (برای بهینه‌سازی)
7         if (log.isDebugEnabled()) {
8             log.debug("Processing user: {}", userId);
9         }
10
11     // با چند پارامتر

```



```

12 String username = "ali";
13 int age = 25;
14 log.info("User: {}, Age: {}", username, age);
15
16 // ↳ exception
17 try {
18     // ...
19 } catch (Exception e) {
20     log.error("Error occurred", e);
21 }
22 }
23 }

```

۱۹.۵ سطوح لاگ (Log Levels)

1			
2			
3		Log Levels (از کمترین به بیشترین)	
4			
5			
6		خطاهای جدی که برنامه رو متوقف می‌کنن ← ERROR	
7			
8			
9		وضعیت‌های غیرعادی ولی قابل بازیابی ← WARN	
10			
11			
12		اطلاعات عمومی (کاربر ثبت شد، سفارش ایجاد شد) ← INFO	

19-Java-Logging

13			
14			
15		اطلاعات برای دیباگ (ورودی متدها، مقادیر) ← DEBUG	
16			
17			
18		جزئی‌ترین اطلاعات (هر قدم کوچک) ← TRACE	
19			
20			
21		هر سطح، سطوح بالاتر از خودش رو هم شامل میشه:	
22		DEBUG و TRACE (نه هست INFO, WARN, ERROR شامل INFO مثلاً)	
23			
24			

مثال: سطوح در عمل

```
1 @Slf4j
2 public class LogExample {
3
4     public void divide(int a, int b) {
5         log.trace("divide called with a={}, b={}", a, b);
6
7         if (b == 0) {
8             log.error("Cannot divide by zero! a={}, b={}", a, b);
9             throw new ArithmeticException("Division by zero");
10        }
11
12        if (a < 0) {
```

19-Java-Logging

```
13     log.warn("Dividing negative number: {}", a);
14 }
15
16 int result = a / b;
17 log.debug("Result: {}", result);
18 log.info("Division completed: {} / {} = {}", a, b, result);
19 }
20 }
```

MDC (Mapped Diagnostic Context) ۱۹.۶

MDC برای ذخیره اطلاعات مرتبط با هر درخواست (مثل User ID, Request ID) مفیده.

```
1 import org.slf4j.MDC;
2
3 @Service
4 @Slf4j
5 public class UserService {
6
7     public void processUser(Long userId) {
8         // MDC ذخیره در ۱.
9         MDC.put("userId", userId.toString());
10        MDC.put("requestId", UUID.randomUUID().toString());
11
12        try {
13            log.info("Processing user");
14            // ...پردازش...
```

19-Java-Logging

```
15     log.info("User processed successfully");
16 } finally {
17     // مهم! MDC پاک کردن
18     MDC.clear();
19 }
20 }
21 }
```

پیکربندی MDC در logback

```
1 <encoder>
2   <pattern>%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - [%X{userId}] [%X{requestId}] -
   %msg%n</pattern>
3 </encoder>
```

Filter برای پر کردن MDC خودکار

```
1 @Component
2 @Order(Ordered.HIGHEST_PRECEDENCE)
3 public class MDCFilter implements Filter {
4
5     @Override
6     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) {
7         try {
8             MDC.put("requestId", UUID.randomUUID().toString());
9             // اگر کاربر لاگین هست
10            Authentication auth = SecurityContextHolder.getContext().getAuthentication();
11            if (auth != null && auth.isAuthenticated()) {
```

```

12     MDC.put("username", auth.getName());
13 }
14     chain.doFilter(request, response);
15 } finally {
16     MDC.clear();
17 }
18 }
19 }

```

۱۹.۷ ! سوالات مصاحبه‌ای

? سوال ۱: SLF4J چیه و چرا ازش استفاده می‌کنیم؟

جواب: SLF4J یه Facade برای کتابخانه‌های لاگ‌گیری هست. مزایا:

- وابستگی به کتابخانه خاصی نداریم
- می‌تونیم کتابخانه لاگ رو عوض کنیم بدون تغییر کد
- API یکسان داره
- پشتیبانی از پارامتریشدن ({})

? سوال ۲: فرق Logback و Log4j2 چیه؟

جواب:

مقایسه Logback و Log4j2

ویژگی	Logback	Log4j2
وضعیت در Spring Boot	پیش فرض Spring Boot	جایگزین Logback
سرعت	سریع	سریع تر
سادگی	ساده تر	پیچیده تر
قابلیت ها	خوب	قابلیت های بیشتر (Async, Lambda) نکات کلیدی:

• Logback به عنوان جانشین Log4j نسخه 1 طراحی شده و به صورت پیش فرض در Spring Boot استفاده می شود.

• Log4j2 نسخه بازطراحی شده Log4j است که از نظر عملکرد (به ویژه با قابلیت Async) از Logback سریع تر است.

• Log4j2 از Lambda Expressions برای Lazy Logging پشتیبانی می کند که می تواند بهبود عملکرد داشته باشد.

• انتخاب بین این دو معمولاً به نیاز پروژه و ترجیح تیم بستگی دارد؛ هر دو کتابخانه قدرتمندی هستند.

? سوال ۳: سطوح لاگ رو به ترتیب از کمترین به بیشترین بگو

جواب:

1 TRACE < DEBUG < INFO < WARN < ERROR

هر سطح، سطوح بالاتر رو هم شامل میشه.

? سوال ۴: چطور لاگ رو در فایل ذخیره کنیم؟

جواب: با تنظیمات logback-spring.xml یا application.properties

1 `logging.file.name=logs/myapp.log`

? سوال ۵: MDC چیه و چه کاربردی داره؟

جواب: MDC (Mapped Diagnostic Context) یه مکانیزم برای ذخیره اطلاعات مرتبط با هر درخواست (مثل User ID, Request ID) در Context لاگ‌گیری هست. این اطلاعات می‌تونن توی همه لاگ‌های اون درخواست ظاهر بشن.